

CHULETA DEFINITIVA — Examen Etapa 1

Imprime esto. Guía completa para el examen del 4 de septiembre.

Incluye: Método de 5 pasos + Fórmulas + Errores comunes + Arq/SO + Planificación.

PÁGINA 1 — MÉTODO Y FÓRMULAS DE LISTAS

MÉTODO DE 5 PASOS — SIEMPRE EN ORDEN

#	Paso	✓
1	Leer el `main` completo: ¿qué recibe cada función? ¿qué debe imprimirse?	☐
2	Tabla de roles: ¿quién es arreglo, contador, dato, índice?	☐
3	Traza en papel: dibuja la tabla de casillas y ejecuta cada vuelta	☐
4	Traducir a código: una línea de traza = una línea de código	☐
5	Compilar, correr y COMPARAR la salida línea por línea con lo esperado	☐

FÓRMULAS DE LISTAS (arreglos + punteros)

Roles de parámetros

"En el arreglo `LLL`, en la casilla `LLL`, guarda `LLL`."

Parámetro	Significado	Ejemplo
<code>`listArray`</code>	El arreglo	<code>`Producto *inventario`</code>
<code>`*listSize`</code>	Contador (lleva `*`)	<code>`int *cantidad`</code>
<code>`data`</code>	Dato nuevo	<code>`int id, char nombre[]`</code>
<code>`index` / `i`</code>	Posición	<code>`int index`</code>

Los 4 patrones

Operación	Corre hacia	Bucle	Cuerpo	Dato va a	Contador
Insertar al inicio	derecha	hacia atrás <code>`i--`</code>	<code>`arr[i] = arr[i-1]`</code>	<code>`arr[0]`</code>	<code>`++`</code>
Insertar después de index	derecha	hacia atrás, para en <code>`i > index`</code>	<code>`arr[i+1] = arr[i]`</code>	<code>`arr[index+1]`</code>	<code>`++`</code>
Eliminar en index	izquierda	hacia adelante desde <code>`index+1`</code>	<code>`arr[i-1] = arr[i]`</code>	(no hay dato)	<code>`--`</code>
Insertar al final	(no corre)	no hay bucle	—	<code>`arr[*listSize]`</code>	<code>`++`</code>

3 reglas de oro

- El destino VIAJA con `i`** — si escribes siempre en la misma casilla, está mal.
- Cero números mágicos** — cada número literal: "*¿de dónde sales?*" → usa el parámetro.
- `index` decide DÓNDE empieza; la aritmética avanza de uno en uno.**

Checklist anti-bug

- ☐ ¿El destino viaja con `i`?
- ☐ ¿Las condiciones son comparaciones con parámetros? (nada de ``i < 5``)
- ☐ ¿El contador se actualiza? (+1 insertar, -1 borrar)
- ☐ ¿Probé con un **SEGUNDO índice** distinto?
- ☐ ¿Comparé salida LÍNEA POR LÍNEA?

PÁGINA 2 — MÉTODO PARA PROBLEMAS NUEVOS (LÓGICA)

CÓMO PENSAR CUANDO NO SABES QUÉ HACER

Cuando el ejercicio NO es igual a los que ya resolviste, sigue estos pasos:

Paso 1: ENTENDER EL PROBLEMA (traducir enunciado)

Responde estas preguntas **en tu papel**:

Pregunta	Respuesta
¿Qué recibe mi función? (¿qué datos entran?)	
¿Qué debe retornar o modificar ? (¿qué sale?)	
¿Qué condiciones hay? (¿qué caso especial existe?)	
¿Qué debe imprimir el `main`?	

Ejemplo `eliminarDuplicados`:

- Recibe: array de Producto, cantidad original, puntero a nuevaCantidad
- Retorna: nuevo array sin duplicados
- Condición: comparar por `id`
- Main imprime: array resultante y nuevaCantidad

Paso 2: PENSAR LA LÓGICA EN PSEUDOCÓDIGO (sin código)

NO escribas código aún. Escribe pasos en español:

```
PARA CADA elemento del array original:
  ¿Ya está en el resultado?
  SI → no lo copio
  NO → lo copio al resultado
```

Guardar cuántos quedan

Paso 3: TRADUCIR A CÓDIGO (paso a paso)

Pseudocódigo	Código C
Para cada elemento	<code>`for (int i = 0; i < cantidad; i++)`</code>
¿Ya está en resultado?	Bucle anidado <code>`for (int j = 0; j < nuevaCantidad; j++)`</code>
Comparar campos	<code>`if (productos[i].id == resultado[j].id)`</code>
Copiar si no está	<code>`resultado[nuevaCantidad] = productos[i]; (*nuevaCantidad)++;`</code>

Patrón 1: FILTRAR (quitar elementos)

Problema: Eliminar duplicados, eliminar por condición, etc.

Lógica:

1. Crear array NUEVO (vacío)
2. Recorrer array original
3. Para cada elemento: ¿cumple condición?
 - SI → copiar al array nuevo, incrementar contador
 - NO → saltar
4. Retornar array nuevo

Template:

```
Tipo *filtrar(Tipo *original, int cantidad, int *nuevaCantidad) {
    Tipo *resultado = malloc(cantidad * sizeof(Tipo));
    *nuevaCantidad = 0;

    for (int i = 0; i < cantidad; i++) {
        if (CONDICION(original[i])) {
            resultado[*nuevaCantidad] = original[i];
            (*nuevaCantidad)++;
        }
    }

    // Redimensionar al tamaño exacto (opcional)
    Tipo *temp = realloc(resultado, *nuevaCantidad * sizeof(Tipo));
    if (temp != NULL) resultado = temp;

    return resultado;
}
```

Ejemplos de condiciones:

- Eliminar duplicados: `!existeEn(resultado, \*nuevaCantidad, original[i])`
- Eliminar por ID: `original[i].id != idAEliminar`
- Filtrar pares: `original[i] % 2 == 0`

Patrón 2: FUSIONAR (combinar dos arrays)

Problema: Unir dos listas, intercalar, mezclar ordenado, etc.

Lógica:

1. Crear array del tamaño suma (cantidad1 + cantidad2)
2. Copiar todos del primero
3. Copiar todos del segundo
4. (Opcional) Ordenar el resultado

Template:

```
Tipo *fusionar(Tipo *arr1, int cant1, Tipo *arr2, int cant2, int *total) {
    *total = cant1 + cant2;
    Tipo *resultado = malloc(*total * sizeof(Tipo));

    // Copiar primero
    for (int i = 0; i < cant1; i++)
        resultado[i] = arr1[i];

    // Copiar segundo
    for (int i = 0; i < cant2; i++)
        resultado[cant1 + i] = arr2[i];

    return resultado;
}
```

Patrón 3: BUSCAR Y MODIFICAR

Problema: Actualizar un elemento, cambiar cantidad, marcar como eliminado, etc.

Lógica:

1. Recorrer buscando por condición (ej: ID)
2. Si lo encuentro → modificar
3. Si no lo encuentro → mensaje de error o retornar -1

Template:

```
int buscarYModificar(Tipo *arr, int cantidad, int idBuscado, ...) {
    for (int i = 0; i < cantidad; i++) {
        if (arr[i].id == idBuscado) {
            // Modificar
            arr[i].campo = nuevoValor;
            return i; // éxito
        }
    }
    return -1; // no encontrado
}
```

Patrón 4: CONTAR/ACUMULAR

Problema: Contar elementos que cumplen, sumar campos, calcular promedio.

Lógica:

1. Inicializar contador/acumulador en 0
2. Recorrer array
3. Si cumple condición → incrementar/sumar
4. Retornar resultado

Template:

```
int contar(Tipo *arr, int cantidad) {
    int contador = 0;
    for (int i = 0; i < cantidad; i++) {
```

```

        if (CONDICION(arr[i]))
            contador++;
    }
    return contador;
}

```

🎯 EJEMPLO APLICADO: eliminarDuplicados

Paso 1 — Entender:

- Recibe: `Producto \*productos`, `int cantidad`, `int \*nuevaCantidad`, `int \*capacidad`
- Retorna: `Producto \*` (nuevo array)
- Lógica: solo quedan productos con IDs únicos

Paso 2 — Pseudocódigo:

```

Crear resultado vacío
PARA CADA producto en original:
    ¿Ya existe uno con mismo id en resultado?
        SI → no copio
        NO → copio, incremento nuevaCantidad
Retornar resultado

```

Paso 3 — Código:

```

Producto *eliminarDuplicados(Producto *productos, int cantidad, int *nuevaCantidad, int *capacidad) {
    Producto *resultado = malloc(cantidad * sizeof(Producto));
    *nuevaCantidad = 0;

    for (int i = 0; i < cantidad; i++) {
        // ¿Ya existe?
        int duplicado = 0;
        for (int j = 0; j < *nuevaCantidad; j++) {
            if (productos[i].id == resultado[j].id) {
                duplicado = 1;
                break;
            }
        }

        // Si no es duplicado, copiar
        if (!duplicado) {
            resultado[*nuevaCantidad] = productos[i];
            (*nuevaCantidad)++;
        }
    }

    return resultado;
}

```

⚠️ ERRORES LÓGICOS COMUNES

Error	Ejemplo	Solución
Olvidar inicializar contador	`int nuevaCantidad;` (basura)	`int nuevaCantidad = 0;`
Comparar arrays completos en vez de campos	`if (productos[i] == resultado[j])`	`if (productos[i].id == resultado[j].id)`
No incrementar contador después de copiar	Falta `(*nuevaCantidad)++`	Siempre después de asignar
Copiar al array original en vez de nuevo	`productos[i] = resultado[...];`	Usar el array NUEVO para guardar
No retornar el array nuevo	Falta `return resultado;`	Siempre retornar

📄 PÁGINA 3 — ERRORES COMUNES EN C

🐛 ERRORES QUE APRENDÍ EN SIMULACROS (ERRORES DE SINTAXIS)

1. `=` vs `==`

```

if (x = 10)    // ASIGNACIÓN → x vale 10, siempre true
if (x == 10)   // COMPARACIÓN → solo true si x ya era 10

```

2. División entera

```

int a = 10, b = 3;
a / b           // → 3 (NO 3.33) – int / int = int
(float)a / b    // → 3.33 – castear ANTES

```

3. Paso por valor (no cambia la variable original)

```

void duplicar(int n) { n = n * 2; }
int a = 5;
duplicar(a);    // a sigue siendo 5 – se copió el valor

```

4. `%s` NO necesita `&`

```

char nombre[] = "Hola";
printf("%s", nombre);    // ✅ correcto
printf("%s", &nombre);   // ❌ incorrecto

```

5. Acceso fuera de rango

```

int nums[] = {10, 20, 30}; // índices válidos: 0, 1, 2
nums[3]    // ❌ fuera de rango

```

6. `scanf` necesita `&` (excepto `%s`)

```

int x;
scanf("%d", &x);    // ✅ necesita &
char s[50];
scanf("%s", s);     // ✅ %s NO necesita &

```

7. Aritmética de punteros

```
int arr[] = {10, 20, 30};
int *ptr = arr; // ptr apunta a arr[0]

*(ptr + 0) // → 10 (arr[0])
*(ptr + 1) // → 20 (arr[1]) ← ¡NO es dirección!
*(ptr + 2) // → 30 (arr[2])
```

8. Confundir parámetros similares

```
void agregar(Producto **inv, int *cantidad, int id, int cant, ...) {
    (*inv)[i].cantidad = *cantidad; // ❌ ERROR: *cantidad es el CONTADOR
    (*inv)[i].cantidad = cant; // ✅ CORRECTO: cant es la cantidad del PRODUCTO
}
```

9. Modificar string literal (segfault)

```
char *str = "Hola";
str[0] = 'J'; // ❌ segfault – literal en memoria de SOLO LECTURA

char str[] = "Hola"; // ✅ array mutable, funciona
str[0] = 'J';
```

10. strcat ACUMULA, no sobrescribe

```
char result[20];
strcpy(result, "Hello"); // result = "Hello"
strcat(result, " "); // result = "Hello "
strcat(result, "World"); // result = "Hello World"
```

PÁGINA 4 — STRUCTS + MEMORIA DINÁMICA

TEMPLATES PARA EXAMEN

Struct básico

```
typedef struct {
    int id;
    char nombre[50];
    int cantidad;
    float precio;
} Producto;
```

malloc — Reservar memoria

```
Producto *inventario = malloc(3 * sizeof(Producto));
if (inventario == NULL) { printf("Error\n"); exit(1); }

inventario[0].id = 1;
strcpy(inventario[0].nombre, "Teclado");
inventario[0].cantidad = 10;
```

realloc — Redimensionar (SIEMPRE con puntero temporal)

```
int nuevaCapacidad = (*capacidad == 0) ? 1 : *capacidad * 2;
Producto *temp = realloc(*inventario, nuevaCapacidad * sizeof(Producto));
if (temp == NULL) { printf("Error\n"); exit(1); }
*inventario = temp;
*capacidad = nuevaCapacidad;
```

⚠️ **BUG DEL ARRANQUE:** Si `*capacidad == 0`, entonces `0*2 = 0` → nunca crece. Usar ternario.

free — Liberar memoria

```
free(inventario);
inventario = NULL; // evitar puntero colgante
*capacidad = 0;
```

Punteros dobles — cuándo usar

Parámetro	Necesito...	Ejemplo
<code>`Producto **inventario`</code>	Modificar el puntero (asignar malloc)	<code>`*inventario = malloc(...)`</code>
<code>`int *cantidad`</code>	Modificar el valor (incrementar)	<code>`(*cantidad)++`</code>
<code>`int id`</code>	Solo leer el valor	<code>`id`</code> directamente

Checklist anti-bug structs

- ☐ ¿Verifiqué ``NULL`` después de ``malloc`/`realloc``?
- ☐ ¿Usé ``strcpy`` para strings (nunca ``=``)?
- ☐ ¿El contador lleva ``(*var)++`` con paréntesis?
- ☐ ¿`realloc` usa puntero temporal?
- ☐ ¿Confundo ``*cantidad`` (contador) con ``cant`` (dato)?

PÁGINA 5 — CORREGIR CÓDIGO "DESORDENADO" (funciona pero imprime mal)

Usalo cuando te den un código que YA compila y corre, pero la salida no está en orden.

Tu tarea: encontrar el/los error(es) de lógica.

MÉTODO PARA ENCONTRAR EL ERROR — hazlo EN ORDEN

Paso 1: ¿QUÉ debería imprimir y QUÉ imprime?

Anota en el papel:

- Salida **esperada** (según el enunciado): `[1,2,3,4,5]`
 - Salida **actual** (como funciona mal): `[3,1,4,2,5]`
- ¿En qué se diferencian? Ese es tu pista inicial.

Paso 2: ESCANEA LAS 5 CAJAS DE BUGS (revisar cada una)

Es casi siempre UNO de estos. Márcalo cuando lo encuentres:

#	Caja	Qué revisar	Señal de error
1	Condición de bucle	`for (i=0; i`	Acceso fuera de rango / falta un elemento
2	Límite / número fijo	`i < 5` en vez de `i < n`	Falla si cambia el tamaño
3	Índice inicial	¿empieza en 0 o en 1? ¿termina en n-1 o n?	Falta el primero o el último elemento
4	Dirección del `for`	¿`i++` debería ser `i--`? ¿recorre al revés?	Elementos invertidos
5	Comparación invertida	`>` cuando debe ser `<` (o al revés)	Orden descendente en vez de ascendente

Paso 3: TRAZA UNA SOLA VUELTA EN PAPEL

Toma la impresión esperada `[1,2,3,4,5]` y el código. Ejecuta **1 o 2 iteraciones** del bucle con números reales. Pregúntate:

"¿Esta vuelta me acerca a lo esperado o me aleja?"

Si la primera vuelta ya produce algo raro, el error está **dentro del bucle** (swap, comparación, asignación).

Paso 4: REvisa el SWAP (cajón común)

El intercambio tiene **3 líneas** y usa una **variable temporal**:

```
//  CORRECTO
int temp = a;    // 1. guardar original de a
a = b;           // 2. a recibe b
b = temp;        // 3. b recibe lo que era a

//  ERRORES FRECUENTES QUE DEJAN TODO REPETIDO O PERDIDO:
a = b;           //  pierde el valor original de a
b = a;           //  ahora a y b tienen lo mismo (temporal mal hecho)
```

Si ves `int temp = a;` pero luego `a = b; b = a;` (sin usar `temp` en el paso 3), **ahí está el bug**: algo se pierde.

LOS 3 DESORDENES MÁS COMUNES Y SU CURA

Tipo A: Los elementos salen al REVÉS (invertidos)

Síntoma: `[5,4,3,2,1]` en vez de `[1,2,3,4,5]`

Causa probable: el `for` recorre al revés, o la comparación está invertida (`>` en vez de `<`).

```
// Cura: bucle hacia adelante + comparación menor
for (int i = 0; i < n; i++)
    for (int j = i+1; j < n; j++)
        if (arr[j] < arr[i]) { int t = arr[i]; arr[i] = arr[j]; arr[j] = t; }
```

Tipo B: Falta el PRIMER o el ÚLTIMO elemento

Síntoma: `[2,3,4,5]` (falta el 1) o `[1,2,3,4]` (falta el 5)

Causa probable: el índice comienza mal (`i=1` en vez de `i=0`) o termina mal (`i < n` en vez de `i <= n`, o un `-1` de más).

```
// Los índices EMPIEZAN en 0 y van hasta n-1
for (int i = 0; i < n; i++)    // 
```

Tipo C: Elementos REORDENADOS pero con % (módulo) mal usado

Síntoma: reagrupa/rota de forma extraña

Causa probable: el índice con `%` o el corrimiento están mal.

```
// Rotación a la izquierda
int temp = arr[0];
for (int i = 0; i < n-1; i++)
    arr[i] = arr[i+1];
arr[n-1] = temp;

//  Error típico: arr[i] = arr[i-1] (recorre mal la dirección)
```

CHECKLIST FINAL ANTES DE ENTREGAR (corrección manual)

- ☐ ¿Los índices van de 0 a `n-1`?
- ☐ ¿Ningún número fijo (`i < 5`) donde va un parámetro (`i < n`)?
- ☐ ¿El swap usa `temp` y en 3 pasos no pierde nada?
- ☐ ¿La comparación coincide con el orden pedido (ascendente = `<`)?
- ☐ ¿El bucle recorre hacia adelante cuando debe ir adelante?
- ☐ ¿Tracé mentalmente la PRIMERA vuelta y es correcta?

PÁGINA 6 — ARQUITECTURA Y SO

DIFERENCIAS QUE CONFUNDO

Data Hazard vs Resource Hazard

Tipo	Qué pasa	Ejemplo
Data hazard	Instrucción necesita resultado de otra que aún no está listo	`R1 = R2 + R3` antes de que `R2` se escriba
Resource hazard	Dos instrucciones necesitan la misma etapa al mismo tiempo	Dos `IF` en el mismo ciclo

Big Endian vs Little Endian

Endian	Dirección baja	Regla
Big Endian	Byte más significativo primero (como se lee)	Motorola, red
Little Endian	Byte menos significativo primero (al revés)	Intel/AMD, x86

Fórmula Little Endian: `Valor = b[0] + b[1]\*256 + b[2]\*65536 + b[3]\*16777216`

Throughput vs Latencia

Concepto	Significado
Throughput	Instrucciones completadas por unidad de tiempo
Latencia	Tiempo que tarda una sola instrucción

Fork vs System Calls

Mecanismo	Para qué sirve
<code>`fork()`</code>	Crear nuevo proceso . AMBOS continúan . Al padre le retorna PID (>0), al hijo le retorna 0.
System calls	Mecanismo para que un proceso solicite servicios del kernel

DMA (Acceso Directo a Memoria)

Permite que **dispositivos de E/S** transfieran datos a/desde memoria **sin intervención de la CPU en cada byte**.

⚠️ DMA **NO** es sobre procesos/memoria virtual. Es sobre **dispositivos E/S**.

FCFS vs HRRN vs SPN

Algoritmo	Tipo	Criterio
FCFS	No expropiativo	El que llegó primero se ejecuta primero
HRRN	No expropiativo	El de mayor ratio (espera + ráfaga)/ráfaga
SPN	Expropiativo*	El de menor ráfaga — minimiza waiting time

⚠️ **Round Robin** es justo en tiempo de respuesta, pero **NO minimiza waiting time**. Eso lo hace **SPN**.

Von Neumann — ¿Dónde está qué?

Componente	Almacena
CPU/Registros	Datos en proceso de uso inmediato
Memoria principal (RAM)	Tanto datos como instrucciones
Disco	Programas y datos a largo plazo

⚠️ En Von Neumann, **datos E instrucciones están en la misma memoria**.

Page Fault

Situación	¿Qué es?
Page fault	Página no está en RAM (está en disco/swap) — el SO la carga
Segmentation fault	Acceso a dirección inválida (bug del programa)

Control Hazard / Branch

¿Qué pasa cuando un **branch es TOMADO**?

El pipeline debe **descartar (flush)** las instrucciones que ya entraron detrás del branch.

Registro Base + Límite vs MMU

Mecanismo	Función
Registro base + límite	PROTEGER : define rango legal de direcciones
MMU / Tabla de páginas	TRADUCIR : convierte direcciones lógicas en físicas

Proceso vs Hilo

Característica	Proceso	Hilo
Espacio de memoria	Propio	Compartido
Stack	Propio	Propio
Creación	<code>`fork()`</code>	<code>`pthread_create()`</code>

📄 PÁGINA 7 — PLANIFICACIÓN DE CPU + NOTAS

📊 PLANIFICACIÓN — SIEMPRE CON GANTT

Regla de oro

🚫 **NUNCA** calcules waiting/turnaround sin dibujar la línea de tiempo primero.

Fórmulas

Turnaround Time = Salida - Llegada
Waiting Time = Turnaround - Ráfaga
Promedio = Suma / Número de procesos

Round Robin — paso a paso

- Escribe la **cola** al inicio: solo los procesos que ya llegaron.
- Saca el primero, ejecuta ``min(quantum, ráfaga)``.
- Si le quedan unidades → **va al fondo de la cola**.
- Si mientras ejecuta **llega un proceso nuevo** → se agrega **después** de los que ya estaban.
- Repite hasta que no queden procesos.

Ejemplo rápido (quantum=4)

P1(0,8) P2(1,4) P3(2,9)

t=0-4: P1 (le quedan 4) Cola: [P2]
t=4-8: P2 (le quedan 0) Cola: [P1, P3]
t=8-12: P1 (le quedan 0) Cola: [P3]
t=12-21: P3 (le quedan 0) Cola: []

⚠️ MIS DEBILIDADES (leer ANTES del examen)

- Números fijos donde va parámetro** → cada número literal: "*¿de dónde sales?*"
- Destino fijo en corrimientos** → en cada vuelta: "*¿esta casilla cambió?*"
- Confiarme con un solo test** → mínimo 2 llamadas distintas.
- `` `` **vs** `` `` → revisar siempre las condiciones.
- División entre `int`** → si necesito decimal, castear ANTES con ``float``.

6. **Cálculo sin Gantt** → SIEMPRE dibujar la línea de tiempo antes de fórmulas.
7. **Conceptos similares** → al responder MCQ, preguntar: "*¿qué lo diferencia exactamente del otro?*"
8. **Parámetros con nombres similares** → `cant` vs `cantidad`: "*¿cuál tiene el dato que necesito?*"
9. **Aritmética de punteros** → `\*(ptr + n)` accede al elemento en posición `n`
10. **Arranque desde capacidad 0** → usar ternario `(capacidad==0)?1:capacidad\*2`
11. **Base/límite vs MMU** → base/límite PROTEGE, MMU TRADUCE
12. **Branch tomado** → se DESCARTA (flush) lo que ya entró
13. **Endian decodificación** → Little: `b0+b1\*256+b2\*65536+b3\*16777216`; Big: al revés
14. **fork() no mata al padre** → AMBOS continúan
15. **SPN vs Round Robin** → SPN minimiza waiting; RR es justo en respuesta
16. **`\*\*pp` desreferencia 2 veces** → primero el puntero, luego el valor
17. **strcat acumula** → nunca sobrescribe
18. **DMA es de dispositivos E/S** → NO es sobre procesos

NOTAS LIBRES (para el día del examen)
